

Not prettier Bash.

IDEL is a policy-aware execution runtime that turns human or AI intent into safe, logged, cross-platform execution. The runtime parses intent, resolves it through a registry, classifies risk, applies policy, runs through an adapter, and writes a signed audit record before anything touches disk.

Why people should care

Safety at the call site

Dangerous intent is explicit, and catastrophic targets such as root, home, drive roots, and raw devices are blocked by default.

Policy before execution

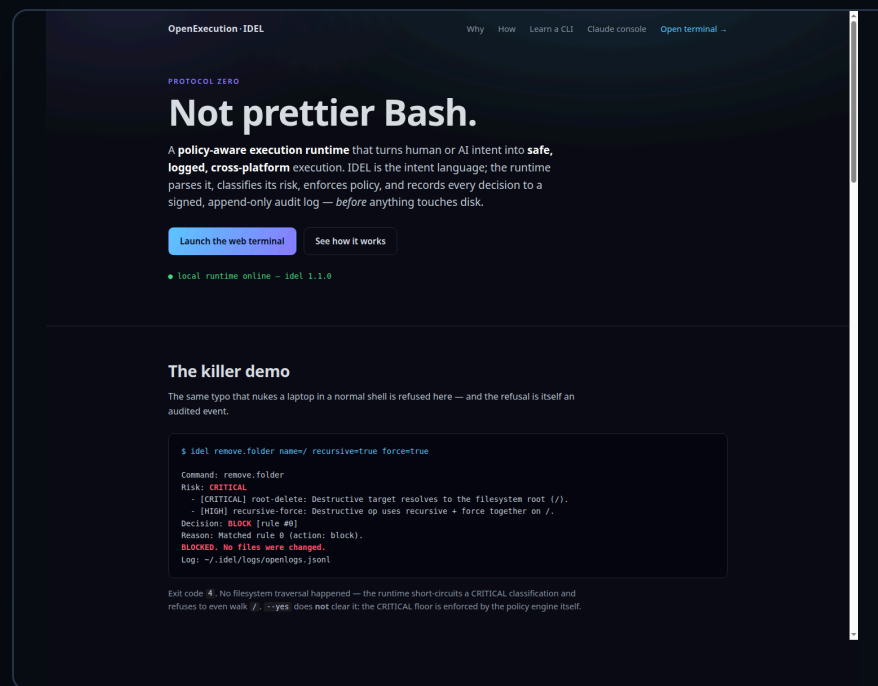
Commands map to allow, warn, dry-run, approval, or block based on risk, command, params, source, and environment.

Audit by design

Every decision, adapter, exit code, duration, and result lands in local signed OpenLogs with secret redaction.

AI stays behind the runtime

ChatGPT or Claude can propose intent, but the runtime remains the enforcement boundary for execution.



The current web landing page, served by the local IDEL runtime.

Same safety pipeline in the browser

The web terminal talks to `idel` serve over loopback. A command typed in the GUI is parsed, risk-scanned, policy-checked, executed or refused, and audited just like the CLI.

```
OpenExecution·IDEL  ● online · idel 1.1.0  IDEL  Ask Claude  Cyan  Menu

Terminal 1  +  sh

idel> show.path
/home/mohammed/Work/openexecution-idel
translates to: pwd
Risk: LOW  · Decision: ALLOW
✓ executed.

idel> remove.folder name=/ recursive=true force=true
translates to: node:remove.folder name=/ recursive=true force=true dryRun=false
Risk: CRITICAL  · Decision: BLOCK
- [CRITICAL] root-delete: Destructive target resolves to the filesystem root (/).
- [HIGH] recursive-force: Destructive op uses recursive + force together on /.
BLOCKED. No files were changed.

idel> explain.registry command=remove.folder
remove.folder (v1.0.0) [core]
Delete a directory, optionally recursively and forcibly.
category: filesystem  riskDefault: HIGH
params:
  name: path (required)
  recursive: boolean (default=false)
  force: boolean (default=false)
  dryRun: boolean (default=false)
safety: {"destructive":true,"targetParam":"name","blockTargets":["/","C:\\","-"],"requiresAffectedPathEstimate":true}
adapters:
  node: @node
    note: Handled in-process by the @node fs adapter so path values are never interpreted as CLI flags.
  posix: rm
  powershell: Remove-Item
    note: Remove-Item -Recurse has historically had quirks with repare points/symlinks (it may traverse them); rm -rf does not follow symlinked dirs into their targets. Treat recursive removal as equally destructive but audit symlink handling per platform.
Risk: LOW  · Decision: ALLOW
✓ executed.

idel> verb.scope param=value  (Tab to complete, ↑↓ history)  RISK  Batch
```

Captured from the live terminal: a LOW-risk command executes, a CRITICAL destructive command is blocked, and the registry explains how `remove.folder` maps across adapters.

CONSOLE PROOF

The dangerous command never runs

IDEL command form

```
$ idel remove.folder name=/ recursive=true force=true

Command: remove.folder
Translates to: node:remove.folder name=/ recursive=true
force=true dryRun=false
Risk: CRITICAL
  - [CRITICAL] root-delete: Destructive target resolves to the
filesystem root (/).
  - [HIGH] recursive-force: Destructive op uses recursive +
force together on /.
Decision: BLOCK [rule #0]
Reason: Matched rule 0 (action: block).
BLOCKED. No files were changed.
Log: ~/.idel/logs/openlogs.jsonl
```

Native shell passthrough

```
$ idel ! rm -rf /

Command: rm -rf /
Translates to: /bin/sh -c rm -rf /
Risk: CRITICAL
  - [CRITICAL] native-rm-rf-root: native `rm` with
recursive+force flags
  - [CRITICAL] native-rm-rf-root-target: native `rm` targeting
root or home directory
Decision: BLOCK [rule #0]
Reason: Matched rule 0 (action: block).
BLOCKED. No files were changed.
Log: ~/.idel/logs/openlogs.jsonl
```

The important claim is narrow and testable: a CRITICAL safety floor cannot be bypassed with - - yes, a lax policy, or native passthrough. The block itself is still recorded as an audit event.

RUNTIME MODEL

One linear pipeline

IDEL is not a shell alias layer. Each command flows through the runtime once, and the runtime decides what can happen.

01

Parse

verb.scope
param=value
becomes a
typed AST.

02

Resolve

Registry lookup
chooses the
command
definition and
adapter set.

03

Classify

AST and
resolved paths
produce LOW,
MEDIUM, HIGH,
or CRITICAL.

04

Policy

Policy maps risk
to allow, warn,
dry-run,
approval, or
block.

05

Execute

Adapters build
real argv or in-
process
operations
without string
templates.

06

Audit

OpenLogs
records the
result, redacted
and signed.

Registry precedence

Content can resolve custom -> official -> core, so a team can teach or shadow commands.

Safety floor precedence

Safety floors resolve core -> everything. A CRITICAL classification cannot be cleared by a custom command, policy file, or AI agent.

How to explain IDEL to ChatGPT or Claude

The clean message for AI users: the model should suggest IDEL intent, not directly own the shell. IDEL then translates the intent to platform execution only after policy and risk checks.

Human or AI
Writes natural intent or an IDEL command.

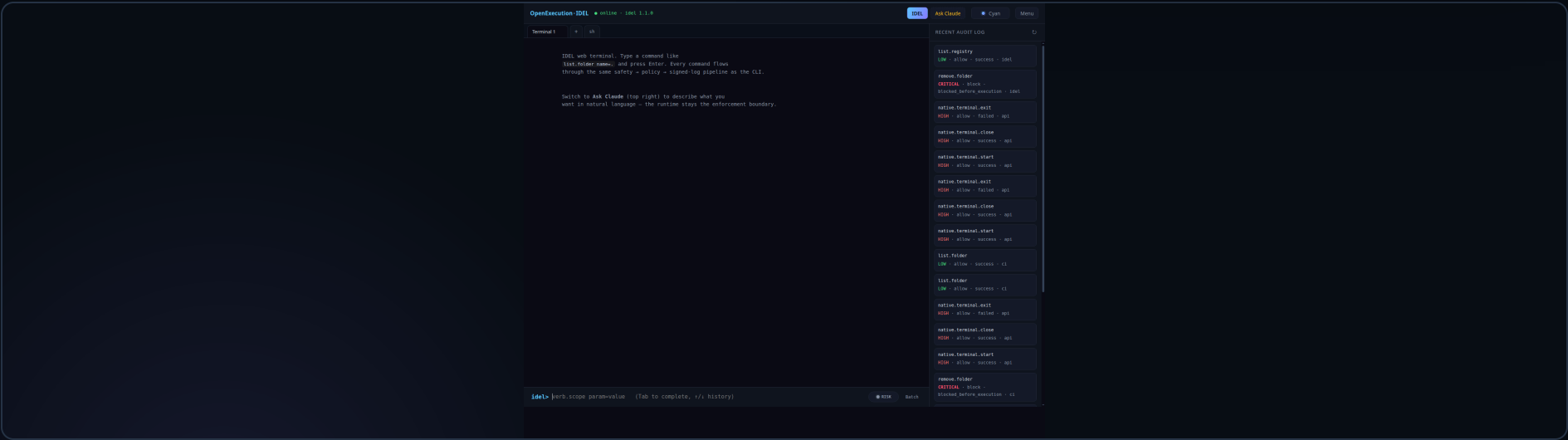
IDEL runtime
Classifies, gates, executes or refuses, then audits.

Operating system
Receives only the allowed adapter invocation.

Pasteable prompt

I am evaluating OpenExecution IDEL, a policy-aware execution runtime for human and AI-proposed commands. Please review it as a safety and developer-experience concept, not as a shell syntax preference.

- Focus on:
- whether the safety/policy/audit runtime boundary is clear;
 - whether CRITICAL floors for commands like root delete are defensible;
 - how ChatGPT or Claude should propose IDEL commands without bypassing the runtime;
 - what examples would best demonstrate the value to developers and teams.



The web terminal also includes Ask Claude mode, native shell tabs, registry completion, workflows, and a visible audit-log panel.

Short sequence for internet posts or calls

Start the UI

```
$ pnpm ui  
  
IDEL Server listening on http://127.0.0.1:7878  
UI serving packages/web/public at http://127.0.0.1:7878/
```

Run the proof

```
$ idel remove.folder name=/ recursive=true force=true  
$ idel ! rm -rf /  
$ idel explain.registry command=remove.folder  
$ idel list.logs
```

What to say

"This is an execution runtime, not just nicer command syntax. It turns intent into a policy decision, then into platform execution, and records the decision."

What to show

Show a safe command succeeding, a destructive command blocked, native passthrough blocked, and the audit trail listing both normal execution and refused execution.

Best first impression: do not lead with "new shell language." Lead with "runtime-enforced safety and auditability for commands proposed by humans or AI agents."